

알약 객체 검출 모델 개발

코드잇 AI 엔지니어 과정 · 초급 프로젝트 최종 보고서

팀 파이리 (1팀)

엄재웅 (팀장) · 신재민 · 김라희 · 윤성원

2026. 09. 10 ~ 2026. 09. 29

목차

1. 요약.....	3
2. 서론.....	4
2.1 프로젝트 배경.....	4
2.2 문제 정의.....	4
2.3 문제의 난이도.....	4
2.4 평가 지표.....	5
2.5 프로젝트 범위 및 제약.....	7
3. 이론적 배경.....	8
3.1 객체 탐지 모델 계열.....	8
3.2 라벨 형식: COCO와 YOLO.....	9
3.3 NMS와 WBF.....	9
3.4 학습률 스케줄.....	10
3.5 클래스 불균형 대응.....	11
3.6 CLAHE 전처리.....	12
4. 데이터 및 전처리.....	13
4.1 원본 데이터 구성.....	13
4.2 어노테이션 파싱.....	13
4.3 결함 데이터 검사 및 제거.....	14
4.4 COCO → YOLO 변환.....	15
4.5 train/val 분할 (근거: 6.3절).....	16
4.6 AI Hub 데이터 통합 (근거: 6.7절).....	16
4.7 오버샘플링 (근거: 6.8절).....	17
5. 실험 설계.....	18
5.1 변인 통제.....	18
5.2 실험 환경 설정.....	18
5.3 실험 기록 자동화.....	19
6. 실험 및 결과.....	20
6.1 기본 설정.....	20
6.2 Epoch 100 & lr 0.001.....	21
6.3 train-val 재분할.....	22
6.4 이미지 사이즈 1280.....	23
6.5 cos_lr.....	24
6.6 하이퍼파라미터의 한계.....	25

6.7 AI Hub 데이터 추가.....	26
6.8 오버샘플링 + WBF 앙상블.....	26
오버샘플링.....	26
WBF 앙상블.....	27
6.9 실패한 실험.....	28
7. 논의.....	29
7.1 튜닝의 문제가 아니라 데이터의 한계였다.....	29
7.2 로컬 검증 점수는 판단 기준이 되지 못했다.....	29
7.3 데이터 수준의 미세조정에는 한계가 있다.....	30
7.4 실패한 실험의 역할.....	30
7.5 남은 실패 유형.....	30
7.6 비즈니스 시사점.....	31
8. 결론 및 향후 과제.....	32
8.1 결론.....	32
8.2 배운 것.....	33
8.3 향후 과제.....	33
9. 참고문헌.....	34
10. 소감.....	35
엄재웅 (팀장).....	35
신재민.....	35
김라희.....	36
윤성원.....	36

1. 요약

본 보고서는 코드잇 AI 엔지니어 과정 초급 프로젝트의 결과로, 이미지 내 여러 알약의 위치와 종류를 동시에 탐지하는 다중 객체 탐지 문제를 다룬다. 평가 지표로 $mAP@[0.75:0.95]$ 를 사용했다. Kaggle에서 제공한 232장·56종을 대상으로 데이터 정제를 수행하고, Faster R-CNN, RetinaNet, YOLO26n을 동일 조건에서 비교하여 YOLO26n을 기본 모델로 선정했다. 이후 해상도·학습률 스케줄·손실 가중치·전이학습·데이터 증강·모델 크기 등 7개 축을 개별로 조정했으나, 제출 점수는 0.39132~0.41738 범위를 벗어나지 못했으며 같은 기간 로컬 검증 mAP 는 0.97 이상이었다.

오류 분석 결과 위치 탐지는 정확한 반면 유사 외형 클래스 간 분류 오류가 주요 실패 유형이었고, 최적화가 아닌 test에 train 56종에 없는 알약이 섞여 있는 정보 부족이 주요 원인으로 확인됐다. AI Hub 「경구약제 이미지 데이터」의 조합 데이터를 train에만 추가해 118종·10,394장으로 확장하자 점수는 0.41550에서 0.61074로 상승했고, 취약 클래스 오버샘플링과 서로 다른 조건으로 학습한 두 모델의 WBF 앙상블을 더해 최종 0.62550을 기록했다. 데이터가 부족한 구간에서는 하이퍼파라미터 탐색의 천장이 낮다는 점, 그리고 로컬 검증 점수가 제출 점수를 대신할 수 없다는 점을 확인했다.

2. 서론

2.1 프로젝트 배경

이 프로젝트는 복약 관리 서비스에서 촬영한 알약 사진을 자동으로 식별하는 상황을 가정한다. 포장에서 분리된 알약은 색과 모양, 표면의 각인만 남는다. 무슨 약인지 알약 데이터베이스와 대조하여 확인해야 한다. 복약 확인 과정을 자동화 할 수 있다면, 확인 및 검수에 드는 시간과 사람의 실수를 줄이는 데 도움을 줄 수 있다.

2.2 문제 정의

입력은 알약 여러 개가 한 평면 위에 놓인 976×1280 크기의 사진이다. 출력은 알약 종류마다 하나씩, 그 알약을 감싸는 박스(바운딩 박스)와 이름, 그리고 모델이 얼마나 확신하는지를 나타내는 신뢰도다. 사진 한 장에 알약이 3~4개씩 들어 있어서, 위치를 찾는 일과 종류를 맞추는 일을 동시에 해야 한다.

받은 데이터는 다음과 같다.

구분	규모	비고
train 이미지	232장	한 장에 3~4알
train 어노테이션	알약별 COCO JSON	박스 좌표, 제품명, 제조사, 각인, 촬영 각도, 앞뒷면
클래스	56종	한 종류당 평균 약 14장 (적은 종류는 3장)
test 이미지	842장	정답 비공개

제출은 CSV 파일로 한다. 찾아낸 박스 하나가 한 줄이 되고, 각 줄에는 이미지 번호, 알약 종류, 박스의 좌표와 크기, 신뢰도가 들어간다. 탐지 위치가 부정확하거나 클래스를 잘못 예측하면 평가 점수에 영향을 준다.

2.3 문제의 난이도

첫째, 알약끼리 생김새가 비슷하다. 보통의 객체 탐지는 사람과 자동차처럼 형태가 뚜렷이 다른 것을 구분한다. 그런데 알약은 색도 모양도 크기도 비슷하고 표면에 새겨진 글자(각인)만 다른 경우가 많다. 콜린알포세레이트 성분의 연질캡슐 3종이 그렇다.

클래스 ID	제품명	제조사	각인
19232	콜리네이트연질캡슐 400mg	한미약품	HM-CN

클래스 ID	제품명	제조사	각인
32310	글리아타민연질캡슐	대웅바이오	글리아타민
18357	종근당글리아티린연질캡슐	종근당	GLT400



셋 다 연한 노란색 타원형 캡슐이다. 각인을 읽지 못하면 사람도 구분하기 어렵다.

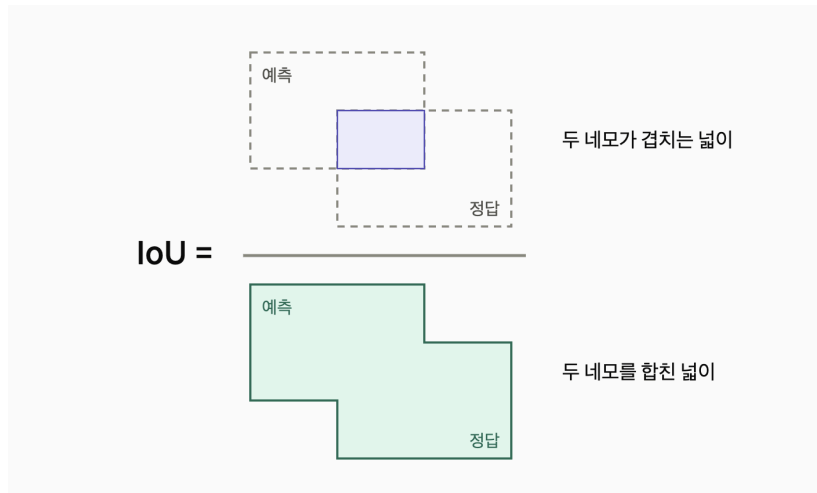
둘째, 그 각인이 늘 보이지는 않는다. 각인은 알약 한쪽 면에만 새겨져 있는데, 데이터는 같은 알약을 앞면과 뒷면 양쪽으로 찍었다(어노테이션의 `drug_dir` 항목). 뒷면이 찍힌 사진에는 구분할 단서가 부족하다. 해상도를 올리거나 학습을 더 시켜도 사진에 없는 글자를 만들어내기 어렵다.

셋째, 알약 종류에 비해 사진이 적다. 232장에 56종이 나뉘어 있어서, 한 알약 종류가 등장한 횟수는 평균 14장 정도이고 적은 것은 3장뿐이다. 각인처럼 작은 차이를 학습하기에는 부족한 양이다.

즉 이 문제에서 어려운 쪽은 "알약이 어디 있는가"가 아니라 "어떤 알약인가"다. 이후 실험에서도 위치 탐지는 비교적 안정적으로 이루어진 반면, 비슷하게 생긴 알약을 헛갈리는 문제는 AI Hub 데이터를 더하기 전까지 계속 남았다.

2.4 평가 지표

프로젝트 점수는 $mAP@[0.75:0.95]$ 다. 여기서 IoU는 모델이 그린 박스와 정답 박스가 얼마나 겹치는지를 0에서 1 사이 값으로 나타낸 것이다.



이 프로젝트는 IoU 0.75부터 0.95까지 0.05 간격으로 다섯 개의 기준을 잡고, 각 기준에서 얻은 점수를 평균한다. 흔히 쓰는 COCO 지표(mAP@[0.50:0.95])와 달리 0.50~0.70 구간은 아예 채점하지 않는다. 박스를 훨씬 정확하게 그려야 점수를 준다는 뜻이다.

예측 하나하나는 다음 세 가지로 판정 된다.

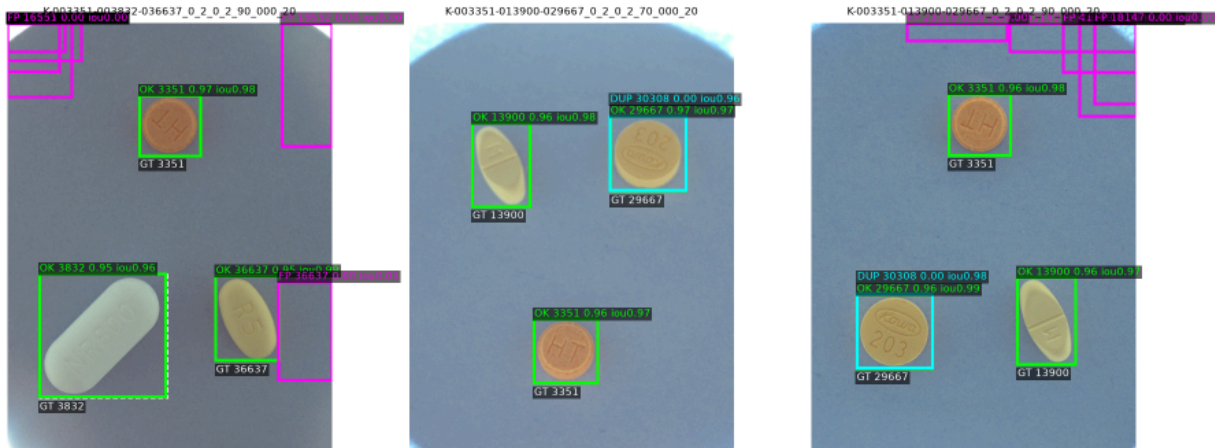
- **TP(맞힘)** — IoU가 기준을 넘으면서 알약 이름도 맞은 경우
- **FP(헛짚음)** — 이름이 틀렸거나 IoU가 기준에 못 미치는 경우
- **FN(놓침)** — 실제로 알약이 있는데 찾지 못한 경우

이 채점 방식은 두 가지를 요구한다.

첫째, 박스를 정확히 그려야 한다. IoU가 0.70인 예측은 위치를 대충 맞혔더라도 최저 기준인 0.75에 못 미쳐서 열 개 구간 어디에서도 정답으로 인정받지 못한다. 오히려 모든 구간에서 헛짚음(FP)으로 처리되고, 그 정답 알약은 놓침(FN)으로 남는다. 그래서 박스 좌표를 평균 내어 오차를 줄이는 WBF 같은 후처리가 이 프로젝트에서는 점수에 직접 반영된다(3.3절).

둘째, 위치와 이름을 함께 맞춰야 한다. 점수는 알약 종류별로 따로 계산한다. 위치를 완벽히 맞혀도 이름이 틀리면 그 예측은 헛짚음이 되고 정답은 놓침으로 남아서, 한 번의 실수가 두 번 반영된다.

이런 이유로 제출할 때는 신뢰도 기준을 **conf=0.001**까지 낮춰 모델이 내놓은 박스를 거의 다 포함시켰다. 점수를 계산할 때 신뢰도가 높은 순서대로 줄을 세우기 때문에, 신뢰도 기준을 낮춰 더 많은 후보를 제출하고, 정답 후보가 평가에 포함될 가능성을 높여, 점수가 오르는 현상을 기대했다. 그래서 결과 이미지를 보면 배경에 쓸데없는 박스가 많이 보이는데, 이는 실수가 아니라 채점 방식에 맞춘 선택이다.



2.5 프로젝트 범위 및 제약

기간과 인원 — 4명, 2026년 9월 10일부터 29일까지 약 3주. 데이터 준비와 전처리, 학습, 평가, 문서 작성을 진행했다.

하드웨어 — 팀원마다 쓰는 환경이 달랐다. 일부는 Apple Silicon(MPS, 메모리 16GB)에서, 일부는 NVIDIA RTX 3060에서 학습했다. MPS 쪽은 메모리 제약으로 해상도와 배치 크기를 함께 올릴 수 없었다. 해상도 1280이면 배치는 4까지, 960이면 8까지가 한계였다. 어느 환경에서 얻은 결과인지 구분하려고 실험 기록에 **device** 항목을 남겼다.

실험 횟수 — MPS 기준으로 한 번 학습하는 데 수십 분이 걸린다. 그래서 시도해 볼 수 있는 조합의 수가 제한됐다. 이 제약 때문에 모델을 고를 때도 성능은 조금 낮더라도 빨리 돌아가는 쪽을 택했다(3.1절). 3.1절의 학습 시간은 모두 MPS에서 같은 조건으로 잰 값이다.

데이터 — 프로젝트가 준 232장 외에 다른 데이터를 써도 된다. 다만 AI Hub 「경약제조합 5000종」 중 TL_2_조합과 TS_2_조합은 프로젝트 제공 데이터와 동일한 원본 데이터이므로 사용하지 않았다.

제출 — Kaggle 제출은 팀 단위로 하루 10번까지다. 실험할 때마다 제출해서 확인할 수 없으니, 어떤 실험을 제출할지 고르는 일 자체가 실험의 일부가 된다.

이러한 제약 속에서 실험을 반복하던 중, 학습 설정을 변경해도 제출 점수가 일정 구간에서 더 이상 상승하지 않는 상황을 마주했다. 이에 다음과 같은 질문을 제기했다.

성능 향상을 가로막는 요인은 학습 설정인가, 데이터인가?

3. 이론적 배경

3.1 객체 탐지 모델 계열

객체 탐지 모델은 "여기 뭔가 있을 것 같다"는 후보 영역을 먼저 뽑는 단계가 있는지에 따라 2단계 방식과 1단계 방식으로 나뉜다.

Faster R-CNN (2단계 방식) — 먼저 사진에서 특징을 뽑고, RPN이라는 부분이 객체가 있을 법한 후보 박스들을 제안한다. 그다음 각 후보 영역의 특징만 따로 떼어내 이름과 정확한 위치를 다시 판별 한다. 후보를 고르고 다시 판별하는 두 번의 과정을 거치므로 그만큼 느리다.

RetinaNet (1단계 방식) — 후보를 따로 뽑지 않고 사진의 모든 위치에서 한 번에 판별한다. 이 방식은 판별 대상 대부분이 배경이라 학습이 배경 쪽으로 치우치기 쉽다. RetinaNet은 Focal Loss로 쉽게 판별되는 배경의 손실 비중을 낮춰, 구분하기 어려운 객체에 학습이 집중되도록 한다. 또 FPN으로 여러 크기의 특징을 만들어 크고 작은 객체를 함께 다룬다.

YOLO (1단계 방식) — 사진을 격자로 나누고 각 칸에서 박스와 이름, 신뢰도를 한 번에 예측한다. 사진 전체를 한 번만 훑기 때문에 가장 빠르다. 같은 알약에 박스가 여러 개 생기면 NMS로 정리한다.

세 모델을 같은 조건(50 epoch, 해상도 640, 배치 4, MPS)에서 직접 학습해 비교했다.

모델	방식	val mAP75-95	학습 시간
Faster R-CNN	2단계	0.8599	61분 8초
RetinaNet	1단계	0.9584	33분 28초
YOLO	1단계	0.9451	14분 37초

본 실험에 사용된 각 모델(Faster R-CNN, RetinaNet, YOLO)은 논문 발표 시점 기준의 최신 공식 소스코드를 바탕으로 비교를 진행했다.

점수만 보면 RetinaNet이 가장 높다. 하지만 YOLO는 RetinaNet 점수의 98.6%를 내면서 학습 시간은 절반에도 못 미쳤다(44%). 같은 시간이면 실험을 두 배 넘게 돌릴 수 있다는 뜻이다. 3주 안에 수십 번 실험해야 하는 상황에서는 한 번의 점수를 조금 더 올리는 것보다 더 많이 시도해 보는 쪽이 낫다고 판단해, YOLO를 기본 모델로 정했다.

3.2 라벨 형식: COCO와 YOLO

원본 정답 파일은 COCO 형식인데 YOLO는 다른 방식을 쓴다.

항목	COCO	YOLO
파일 형태	데이터 전체가 하나의 JSON	이미지 1장당 .txt 1개
좌표 기준	박스의 왼쪽 위 모서리 (x, y)	박스의 한가운데 (cx, cy)
좌표 단위	실제 픽셀값	0~1 사이 비율
클래스 번호	알약 코드 그대로 (1900, 2483... 비 연속적)	0부터 연속적인 번호

YOLO 라벨은 한 줄에 알약 하나를 클래스 번호 cx cy w' h' 형식으로 적는다. 따라서 변환 과정에서는 박스 좌표를 바꾸는 일과 클래스 번호를 정하는 일을 함께 해야 한다.

먼저 좌표는 픽셀 단위 (x, y, w, h)를 사진 크기에 대한 비율로 바꾼다. W와 H는 원본 사진의 가로, 세로 길이다.

$$\begin{aligned} cx &= (x + w/2) / W & w' &= w / W \\ cy &= (y + h/2) / H & h' &= h / H \end{aligned}$$

다음으로 각 박스에 붙일 클래스 번호를 정한다. 재현이 가능하려면 누가 실행해도 같은 번호가 나와야 한다. 그래서 알약 ID를 작은 순서대로 정렬한 뒤 0부터 차례로 번호를 붙였다. 정렬을 기준으로 삼았으니 실행하는 사람이나 환경이 달라도 번호는 바뀌지 않는다.

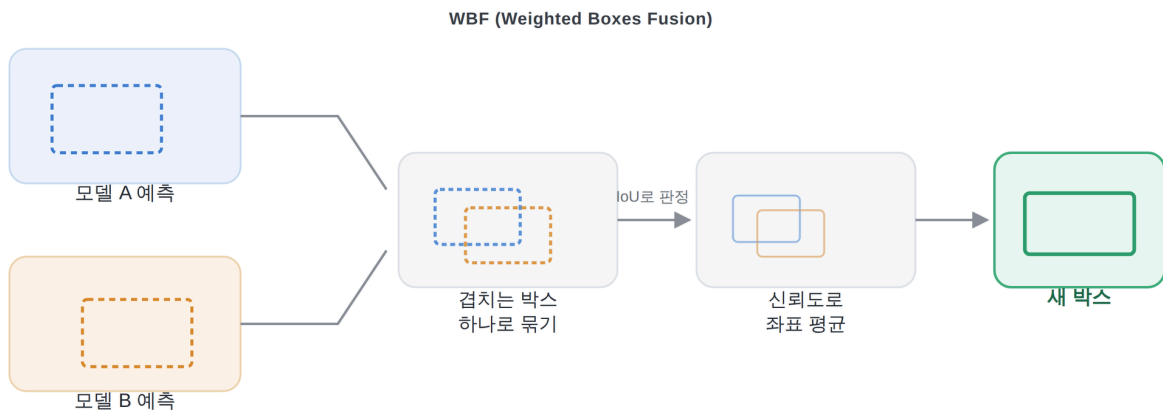
3.3 NMS와 WBF

같은 알약에 박스가 여러 개 겹쳐 나올 때 정리하는 방법은 두 가지다.

NMS — 겹치는 박스 중 신뢰도가 가장 높은 하나만 남기고 나머지는 버린다. 버린 박스가 가지고 있던 위치 정보도 같이 사라진다.

WBF (Weighted Boxes Fusion, Soloviyev et al., 2021) — 많이 겹치는 박스들을 한 덩어리로 묶고, 좌표와 점수를 신뢰도에 따라 평균 내어 **새 박스를 만든다**. 버리는 대신 합치는 방식이다.

```
모델 A 예측 ┌
              ├── 많이 겹치면 한 덩어리로 — 신뢰도로 평균 — 새 박스
모델 B 예측 └
```



평균을 내면 한쪽으로 치우친 오차가 서로 상쇄된다. 이 프로젝트는 **IoU 0.75~0.95** 구간을 채점하므로 박스가 조금만 정확해져도 점수에 바로 반영된다. 그래서 **WBF**가 이 프로젝트에 가장 적합한 방법이라고 판단했다.

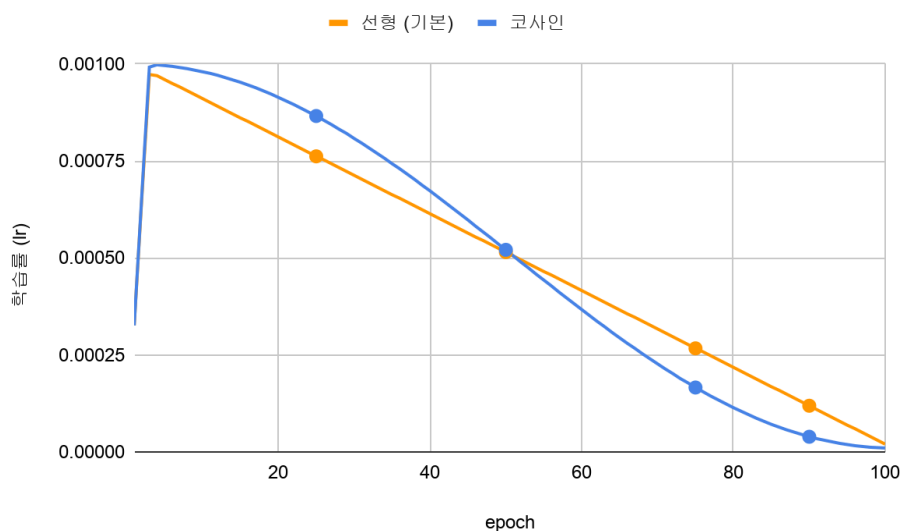
단, 여기에는 전제가 하나 있다. 합칠 예측들이 서로 다른 곳에서 틀려야 한다. 그래야 평균을 낼 때 오차가 상쇄된다. 같은 모델에 입력만 뒤집거나 크기를 조금 바꿔 여러 번 돌리는 방식은 이 조건을 채우기 어렵다. 이러한 미세한 차이는 모델이 학습 중에 이미 본 범위라 거의 같은 답이 나오고, 같은 곳에서 틀린 답끼리는 아무리 합쳐도 새로 얻는 정보가 없다. 이 예상은 **6.9절** 실험에서 실제로 확인했다.

3.4 학습률 스케줄

학습률은 모델이 한 번에 가중치를 고치는 양으로, 보폭에 비유할 수 있다. 보폭이 크면 빨리 움직이지만 목적지 앞에서 정확히 서기 어렵고, 작으면 정밀하지만 느리다. 그래서 학습이 진행될수록 보폭을 줄여 나간다.

Ultralytics의 기본값은 보폭을 일정한 속도로 줄이는 방식(선형 감쇠)이고, 최종적으로 처음의 1%까지 내려간다. `cos_lr=True`로 바꿔도 두 방식은 같은 값에서 시작해 거의 같은 값으로 끝난다. 달라지는 것은 그 사이에서 **학습률이 줄어드는 곡선의 모양**이다.

epoch	선형 (기본)	코사인
25	0.000762	0.000866
50	0.000515	0.000521
75	0.000267	0.000166
90	0.000119	0.000039



표와 그래프를 보면 코사인은 50 epoch까지 선형보다 큰 학습률을 유지하다가, 50 epoch부터 더 빠르게 줄어 80 epoch 이후에는 선형의 절반 아래로 내려간다. 초반에는 큰 보폭으로 넓게 움직이고, 후반에는 작은 보폭으로 세밀하게 다듬는 셈이다. 표와 그래프의 값은 두 실험의 `results.csv`에 실제로 기록된 값이다(시작 학습률 0.001, 100 epoch 기준).

3.5 클래스 불균형 대응

클래스 불균형은 어떤 알약의 학습 사진만 유독 적은 상황을 말한다. 이러면 모델은 사진이 많은 알약은 잘 학습하지만 적은 알약은 제대로 학습하지 못한다. 이번에 검토한 대응 방법은 두 가지다.

오버샘플링 — 학습이 부족한 알약의 사진을 학습 중에 더 자주 보여주는 방법이다. 새 사진을 만드는 게 아니라 있던 사진을 여러 번 보여주는 것이다. 이번에는 사진 파일을 실제로 복사하는 대신, 같은 파일을 가리키는 이름만 하나 더 만드는 하드링크를 써서 디스크 용량이 늘지 않게 했다. 다만 같은 사진이 반복되므로 배율을 높일수록 그 사진에만 익숙해지는 과적합 위험이 커진다.

cls_pw — 이름을 틀렸을 때 주는 벌점의 세기를 조절하는 값이다. 이 값을 바꾸면 사진이 적은 알약에 모델이 얼마나 민감하게 반응할지가 달라진다.

3.6 CLAHE 전처리

CLAHE(Contrast Limited Adaptive Histogram Equalization)는 사진을 작은 타일로 나누고, 타일마다 따로 밝기 대비를 키우는 전처리다. 사진 전체에 같은 보정을 하는 대신 부분마다 따로 보정하기 때문에, 배경과 구분이 잘 되지 않던 각인이 선명하게 보일 것으로 기대했다.

하지만 배경처럼 원래 밝기가 높은 부분에서도 작은 차이까지 키우기 때문에 없던 얼룩이 생길 수 있다. 이번 데이터에서는 이 부작용이 더 크게 작용해 오히려 성능이 떨어졌다(6.9절).

4. 데이터 및 전처리

원본 232장 중 문제가 있는 12장을 제외하여 220장을 확보했다. 나중에 AI Hub 데이터를 추가하여 학습 사진을 10,394장, 알약을 118종으로 확대했다.

4.1부터 4.4까지는 프로젝트 초기에 진행한 기본 전처리 단계이며, 4.5부터 4.7까지는 실험하는 과정에서 발견한 문제를 해결하기 위해 추가한 전처리다. 각 단계를 추가한 이유는 6장의 실험 결과에서 다룬다. 이 장은 최종 데이터가 어떻게 만들어졌는지 한눈에 확인할 수 있도록 정리했다.

4.1 원본 데이터 구성

구분	내용
train_images	232장, 976×1280, 한 장에 3~4알
train_annotations	알약별 COCO JSON. 사진 한 장에 찍힌 알약 수만큼 파일이 있음
test_images	842장, 정답 비공개

정답 파일에는 박스 좌표 말고도 제품명(dl_name), 주성분(dl_material), 제조사(dl_company), 각인(print_front), 촬영 각도(camera_la), 앞뒷면(drug_dir), 배경색, 조명색 같은 정보가 함께 들어 있다. 이 중 print_front와 drug_dir은 나중에 모델이 왜 헛갈렸는지 밝혀내는 데 단서로 활용했다.

4.2 어노테이션 파싱

정답 파일은 사진 한 장에 찍힌 알약 수만큼 JSON으로 나뉘어 있다. train_annotations 아래의 모든 JSON을 돌면서 사진 정보, 박스 좌표, 알약 종류를 꺼내고, 사진 파일 이름을 기준으로 한 장에 속한 알약들을 하나로 묶었다.

```
for f in raw.glob("train_annotations/**/*.json"):
    data = json.loads(f.read_text(encoding="utf-8"))
    img = data["images"][0]
    x, y, w, h = data["annotations"][0]["bbox"]
    cat = data["categories"][0]

    by_image[img["file_name"]].append(
        {"x": x, "y": y, "w": w, "h": h,
         "class_id": cat["id"], "class_name": cat["name"]}
    )
```

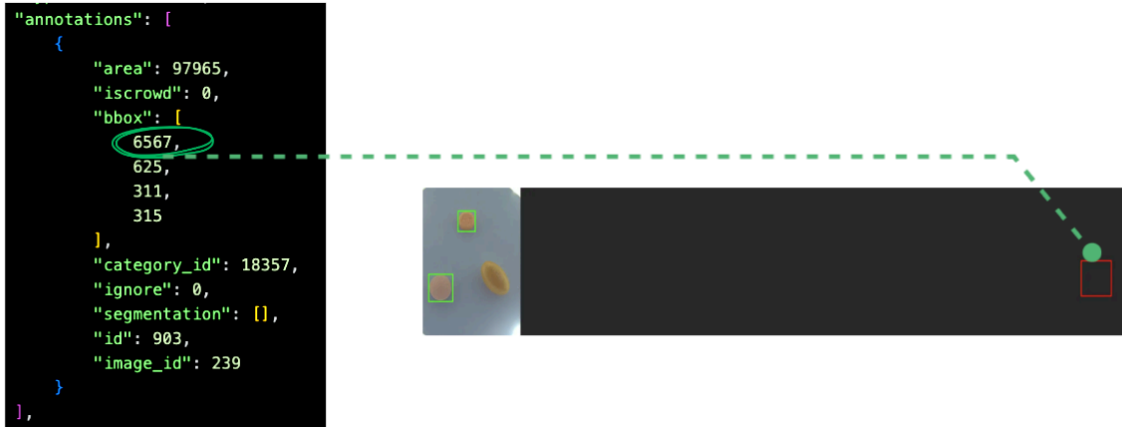
이후의 모든 검사와 변환은 이렇게 묶은 데이터를 대상으로 한다. 사진 한 장의 알약들을 한자리에 모아야 전체 라벨의 누락이나 중복 오류를 사진 단위로 검증할 수 있기 때문이다.

4.3 결함 데이터 검사 및 제거

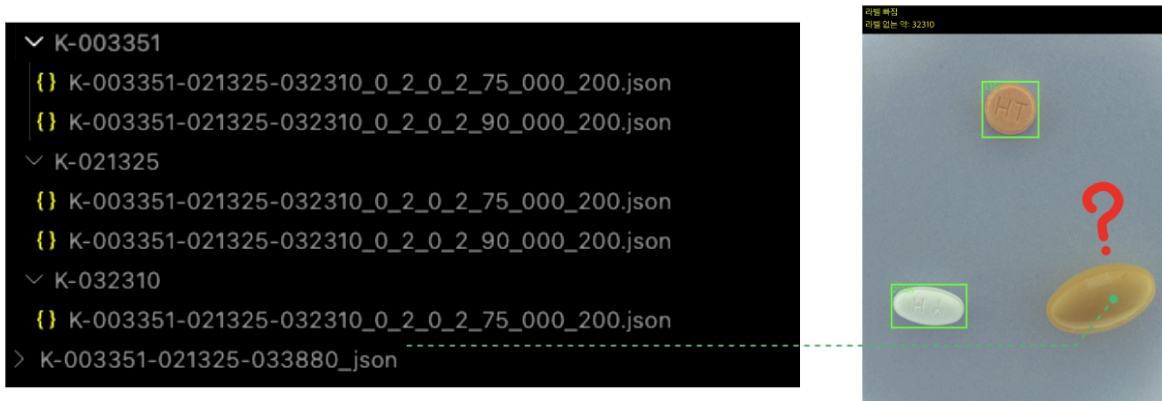
세 가지를 차례로 검사했다. 문제가 된 박스만 제외하는 대신 **사진 자체를 제외했다**. 정답이 사라진 알약이 사진에 그대로 남아 모델이 배경의 일부로 오인하여 잘못된 학습을 방지하기 위함이다.

검사	판정 기준	제외	남은 장수
박스 범위	좌표가 사진 밖이거나 가로·세로가 0 이하	1장	231
라벨 누락	파일 이름에 있는 약 코드의 JSON이 없음	8장	223
라벨 중복	같은 박스에 서로 다른 알약 이름이 붙음 (IoU 0.9 이상)	3장	220

박스 범위 — image_id 239의 정답 파일은 박스의 x좌표가 6567이었다. 원본 사진의 가로 길이가 976이니 범위를 벗어난 값이다.



라벨 누락 — K-003351-021325-032310 조합에서 카메라 위도 90도로 찍은 K-032310의 JSON이 없었다. 사진에는 알약이 세 개 있는데 정답은 두 개뿐이다.



라벨 중복 — image_id 1228에서 똑같은 좌표 [88, 255, 366, 209]에 신바로정(33009)과 가바토파정 100mg(16548)이 동시에 붙어 있었다. 알약 하나에 정답이 두 개이므로 학습에 방해가 된다. 두 박스가 같은 것인지는 IoU로 판정했다.



4.4 COCO → YOLO 변환

3.2절에서 본 형식 차이 중 실제로 코드를 짜야 하는 건 두 가지다.

좌표 변환 — 왼쪽 위 모서리 기준인 (x, y)를 한가운데 (cx, cy)로 옮기고, 사진 크기로 나눠 0~1 사이 값으로 바꾼다.

```
def to_yolo_line(b, idx):
    cx = (b["x"] + b["w"] / 2) / IMG_W
    cy = (b["y"] + b["h"] / 2) / IMG_H
    w = b["w"] / IMG_W
    h = b["h"] / IMG_H
    return f"{idx} {cx:.6f} {cy:.6f} {w:.6f} {h:.6f}"
```

클래스 번호 다시 매기기 — 원본의 알약 코드는 1900, 2483, 3351처럼 비연속적으로 떨어져 있어, YOLO가 그대로 쓸 수 없다. 등장하는 알약 ID를 오름차순으로 정렬한 뒤 0부터 번호를 붙이고, 그 대응 관계를 data.yaml에 적어줬다.

```
names:
  0: '1900'
  1: '2483'
  2: '3351'
  ...
```

정렬 기준을 오름차순으로 고정했기 때문에 누가 실행해도 같은 번호가 나온다. 예측 결과를 제출할 때는 이 대응 관계를 거꾸로 적용해 YOLO 번호를 원래 알약 코드로 되돌린다. 변환하고 나면 사진 한 장당 .txt 파일 하나가 생기고, 파일의 한 줄에는 박스 하나의 정보가 담긴다.

4.5 train/val 분할 (근거: 6.3절)

처음에는 학습할 사진과 검증할 사진을 무작위로 나눴다. 그런데 두 가지 문제가 생겼다.

첫째, 어떤 알약이 학습에서 통째로 빠진다. 클래스 33009가 학습 데이터에 한 장도 들어가지 않아서, 검증할 때 그 알약에 대한 예측이 아예 나오지 않았다. 등장하는 사진이 3장뿐인 알약도 있어서, 무작위로 나누면 한 알약이 전부 검증 쪽으로 몰릴 수 있다.

둘째, 검증 점수가 실제보다 높게 나온다. 같은 알약 조합을 카메라 위도 70도, 75도, 90도로 찍은 사진이 학습과 검증에 나뉘어 들어가면, 모델은 학습에서 이미 본 것과 거의 똑같은 사진으로 시험을 보는 셈이 된다.

그래서 나누는 기준을 사진에서 알약 조합으로 바꿨다. 같은 조합의 사진은 모두 학습이나 검증 한쪽에만 들어간다. 다만 이렇게 나누면 어떤 알약이 검증에만 있고 학습에는 없는 경우가 생길 수 있다. 그래서 나눈 뒤 학습에 없는 알약이 있으면 그 알약이 포함된 조합을 검증에서 학습으로 옮기는 보정 과정을 넣었다. 처음에는 검증 비율을 20%로 나눴고, 보정을 거친 최종 결과는 학습 183장, 검증 37장(약 17%)이다.

4.6 AI Hub 데이터 통합 (근거: 6.7절)

AI Hub 「경구약제 이미지 데이터」(datasetkey 576)의 조합 데이터를 학습 데이터에 더해 알약 종류를 확장했다.

항목	통합 전	통합 후
알약 종류	56종	118종 (+62)
학습 이미지	183장	10,394장 (Kaggle 183 + AI Hub 10,211)
검증 이미지	37장	37장 (그대로)

데이터 확장 과정에서 다음의 두 가지 원칙을 준수했다.

검증 데이터는 Kaggle 원본만 유지했다. 검증 쪽이 바뀌면 이전 실험과 점수를 비교할 수 없기 때문이다.

프로젝트 원본 데이터는 쓰지 않았다. AI Hub 「경구약제조합 5000종」 중 TL_2_조합(66066)과 TS_2_조합(66155)은 이번 프로젝트의 학습·평가 데이터의 원본에 해당한다. 이 데이터를 학습에 넣으면 시험 문제의 답안을 보는 것과 같아 점수가 높게 나올 것이다. 다운로드 스크립트 단계에서 두 파일을 아예 제외하도록 만들었다.

한 가지 걸림돌이 있었다. AI Hub 정답 파일은 알약 종류 번호(category_id)가 전부 1로만 적혀 있어 구분에 쓸 수 없다. 대신 drug_N 항목(예: K-019232)을 알약 식별자로 써서 Kaggle 데이터의 알약 코드와 맞췄다.

데이터가 약 56배로 늘어 한 epoch에 들어가는 사진 수도 그만큼 많아졌으므로, 학습 횟수(epoch)를 100에서 30으로 줄였다. 이때 warmup_epochs와 close_mosaic도 같은 비율로 줄였다. 두 값은 epoch 수로 정해지기 때문에, 전체 epoch만 줄이고 그대로 두면 학습 전체에서 차지하는 비중이 커진다. 예를 들어 close_mosaic을 10으로 두면 100 epoch에서는 마지막 10%만 모자이크를 끄지만, 30 epoch에서는 마지막 3분의 1을 끄게 된다. 학습 초반의 적응 구간과 마지막 마무리 구간의 비중을 이전 실험과 같게 유지하려고 비율을 맞췄다.

4.7 오버샘플링 (근거: 6.8절)

알약별 점수를 확인해 보니 유독 한 종류만 뒤쳐지는 경우가 있었다. 그 알약이 찍힌 사진을 복제해 학습 중에 더 자주 보여줬다.

복제는 하드링크로 했다. 파일 시스템이 같은 파일을 가리키는 이름을 하나 더 만드는 방식이라 디스크 용량이 늘지 않고 사진 내용도 원본과 똑같다. 새 사진을 만들어내는 증강과 달리 **있던 사진을 더 자주 보여주는 것**이라 사진의 다양성 자체는 늘지 않는다. 그만큼 그 사진에만 익숙해질 위험이 있어 배율을 임의로 높일수없고, 대상 알약과 배율은 실험으로 정했다.

5. 실험 설계

5.1 변인 통제

실험은 **한 번에 하나만 바꾼다**는 원칙으로 진행했다. 두 가지를 동시에 바꾸면 점수가 움직여도 무엇 때문인지 알 수 없고, 결국 실험을 다시 해야 하기 때문이다.

기준이 되는 설정은 고정된 게 아니라 실험이 진행되면서 계속 갱신됐다. 어떤 실험이 점수를 올리면 그 설정이 다음 실험의 기준이 되고, 거기서 다시 하나만 바꾼다. 그래서 6장의 각 절은 바로 앞 절의 설정을 물려받은 상태에서 한 가지만 달라진 결과다.

무엇을 바꿨는지는 실험 이름에 그대로 드러나게 규칙을 정했다. 폴더 이름만 봐도 누가 무엇을 바꾼 실험인지 알 수 있어서, 4명이 동시에 실험할 때 같은 걸 두 번 하는 일을 막아줬다.

```
jw_ep100_img1280_batch4_lr0.001_cos_lr
└─ 작성자_학습횟수_해상도_배치_학습률_바꾼점
```

5.2 실험 환경 설정

팀원 모두가 같은 조건에서 실험해야 결과를 공정하게 비교할 수 있다. 그래서 같은 설정으로 다시 실행하면 언제나 같은 결과가 나오도록 다음 네 가지를 설정했다.

무작위 고정 — SEED = 42, deterministic: True. 데이터를 나누는 방식, 처음 가중치를 정하는 방식, 증강의 무작위성이 모두 이 값을 따른다.

설정값 자동 저장 — 학습이 끝나면 runs/<NAME>/args.yaml에 실제로 쓰인 설정이 전부 기록된다. 사람이 적어둔 메모가 아니라 프로그램이 남긴 값이라, 나중에 "이 실험이 정확히 어떤 조건이었나"를 두고 헛갈릴 일이 없다.

데이터셋 전환 스위치 — src/config.py의 USE_AIHUB 한 줄로 56종과 118종 데이터를 오간다. AI Hub를 도입한 뒤에도 그 이전 실험을 그대로 다시 돌릴 수 있다.

환경 고정 — uv로 패키지 버전을 잠그고 uv.lock을 저장소에 올렸다. 팀원은 uv sync 한 줄로 똑같은 환경을 만든다.

데이터는 용량이 커서 저장소에 넣지 않고, 원본에서 학습용 데이터를 다시 만드는 스크립트를 제공했다. 생성 후에는 check_yolo로 사진과 라벨의 짝을 검사하며, 최종 데이터의 "라벨 11,239개(오버샘플링 복제 808개 포함) 중 어긋난 것은 0개였다."

```
uv sync # 환경 재현
uv run python -m src.data.download_aihub # AI Hub 데이터 (선택)
uv run python -m src.data.make_yolo # COCO → YOLO 변환
uv run python -m src.data.check_yolo # 라벨 검증
uv run python -m src.train.train_yolo # 학습
uv run python -m src.analysis.predict # test 예측 → 제출 파일
```

5.3 실험 기록 자동화

4명이 3주 동안 수십 번 실험을 돌리면 기록이 흩어진다. 어떤 설정이 몇 점이었는지가 각자 메모에만 남으면 비교할 수 없고, 같은 실험을 두 번 하게 된다. 그래서 기록을 자동으로 이어지게 만들었다.

단계	실행 주체	결과
1	train_yolo.py	학습 종료, 설정값·점수 확보
2	Apps Script 웹앱	구글 시트에 실험 한 줄 추가
3	GitHub Actions	시트를 읽어 README 점수 그래프 다시 그림
4	GitHub Actions	docs/readme-graph 브랜치로 PR 생성
5	팀장	확인 후 머지

시트에 들어가는 항목 중 설정값과 검증 점수는 자동으로 채워지고, 제출 점수와 결론·다음 실험 방향은 직접 적는다.

자동화 범위를 일부러 여기까지만 뒀다. 기록하고 모으고 문서를 갱신하는 일은 자동으로 하되, **다음 실험을 무엇으로 할지는 사람이 정한다**. 점수만 보고 기계적으로 다음 설정을 고르면 6.6절에서 다루는 "0.41의 벽" 같은 구조적인 문제를 알아채지 못하고 같은 자리를 맴돌게 되기 때문이다.

[illegible]

구글 시트 주소와 AI Hub API 키는 .env와 저장소 Secret에만 두고 코드에 올리지 않았다.

6. 실험 및 결과

실험은 앞 실험에서 틀린 사례를 보고 왜 틀렸을지 가정한 다음, 설정 하나를 바꾸는 식으로 진행했다. 각 절은 관찰 → 가설 → 조치 → 결과 순서로 정리했다. 전체 흐름은 0.235552에서 0.62550까지다.

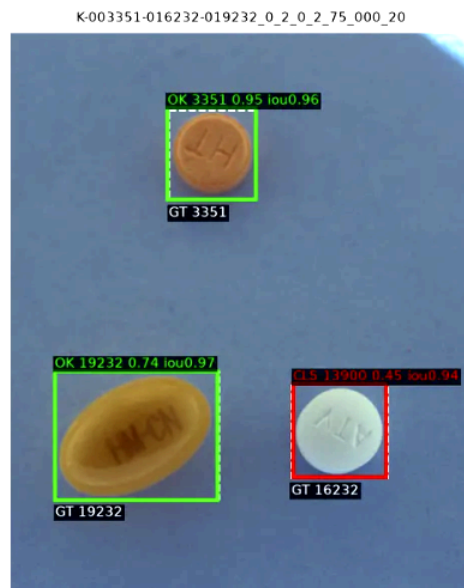
6.1 기본 설정

설정 — YOLO26n, epoch 50회, 해상도 640, 배치 16. 미리 학습된 가중치에서 시작해 별다른 조정 없이 학습했다.

결과 — 검증 점수 0.7179 / 제출 점수 **0.235552**

관찰 — 틀린 사례를 보니 위치는 잘 찾는데 이름을 헛갈리는 게 눈에 띄었다. 16232 알약은 박스를 거의 정확히 그렸는데도(loU 0.94) 비슷하게 생긴 13900으로 답했고, 그마저도 신뢰도가 0.45로 낮았다.

항목	값
정답	16232
예측	13900 (틀림)
IoU	0.94
신뢰도	0.45



알약을 못 찾은 게 아니라 어떤 알약인지 고르지 못한 것이다. 2.3절에서 예상한 어려움이 첫 실험에서 그대로 나타났다.

6.2 Epoch 100 & lr 0.001

가설 — 두 가지를 의심했다. 하나는 해상도다. 원본이 976×1280인데 640으로 줄이면 각인처럼 작은 무늬가 뭉개진다. 생김새가 같은 알약은 각인으로만 구분되니 해상도를 올리면 나아질 것으로 봤다. 다른 하나는 학습량이다. epoch 50회 지점에서도 검증 점수 그래프가 아직 올라가는 중이었다.

조치 — 해상도 640 → 960, epoch 50회 → 100회, 옵티마이저 AdamW, 학습률 0.001

결과 — 검증 점수 0.7179 → **0.9321** / 제출 점수 0.235552 → **0.39139**

두 점수 다 크게 올랐다. 해상도와 학습량을 동시에 바꿨기 때문에 각각이 얼마나 기여했는지는 알 수 없다. 다만 이 시점의 목표는 원인을 밝히는 게 아니라 쓸 만한 출발점을 만드는 것이었다.

관찰 — 남은 실패는 두 종류였다.

정답	현상	상세
19607	이름 틀림	위치는 정확한데(loU 0.98) 14232로 답함, 신뢰도 0.42
33009	아예 못 찾음	예측한 박스가 하나도 없음



33009는 신뢰도가 낮은 게 아니라 박스 자체가 없었다. 단순히 학습이 부족해서라고 보기 어려운 현상이라 데이터를 열어봤고, 그 결과가 다음 절이다.

6.3 train·val 재분할

관찰 — 6.2에서 33009를 못 찾은 이유를 추적했더니, **33009는 학습 데이터에 한 장도 없었다**. 배운 적이 없으니 맞힐 수 없는 게 당연했다.

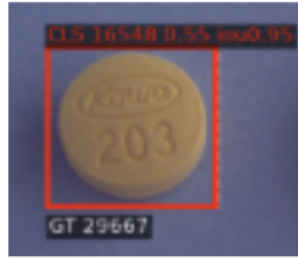
가설 — 사진을 무작위로 나눈 게 원인이다. 등장하는 사진이 몇 장뿐인 알약은 통째로 검증 쪽으로 빠질 수 있다.

조치 — 나누는 기준을 사진에서 알약 조합으로 바꾸고, 학습 쪽에 없는 알약이 생기면 그 조합을 검증에서 학습으로 되돌리는 보정을 넣었다(4.5절).

결과 — 검증 점수 0.9321 → **0.984** / 제출 점수 0.39139 → **0.41503**

관찰 — 알약을 못 찾는 문제는 사라지고 이름이 틀리는 문제만 남았다. 29667(리바로정)을 비슷하게 생긴 16548(가바토파정)으로 답했다. 위치는 정확했고(IoU 0.95) 신뢰도는 0.55였다.





리바로정



가바토파정

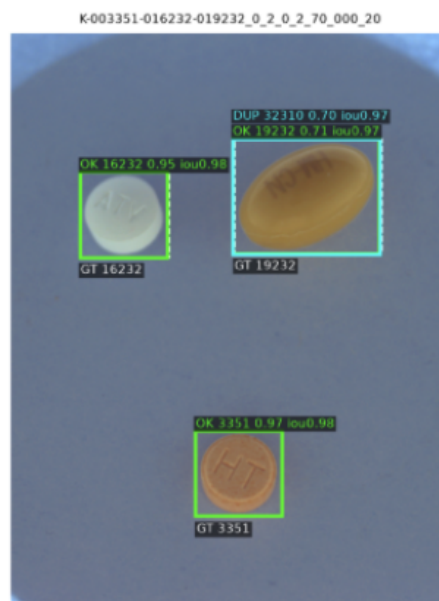
6.4 이미지 사이즈 1280

가설 — 리바로정과 가바토파정은 모양도 색도 크기도 비슷해서 각인으로만 구분된다. 960px에서는 각인이 몇 픽셀로만 표현되니, 해상도를 더 올리면 각인을 알아볼 수 있을 것으로 봤다.

조치 — 해상도 960 → 1280. 메모리가 부족해 배치는 8 → 4로 줄였다.

결과 — 검증 점수 0.984 → **0.984** / 제출 점수 0.41503 → **0.40805**

관찰 — 예상대로 이름을 틀리거나 못 찾는 문제는 사라졌다. 앞 절에서 틀렸던 29667도 제대로 맞췄다. 그런데 제출 점수는 오히려 내려갔다.



남은 실패는 하나였다. 19232 알약에 정답을 맞힌 박스(신뢰도 0.71, IoU 0.97)와 함께, 같은 자리에 32310이라고 답한 박스(신뢰도 0.70, IoU 0.97)가 하나 더 붙었다. 모델이 두 알약 사이에서 거의 같은 확신으로 결정을 내리지 못한 것이다. 19232와 32310은 2.3절에서 본 18357과 함께 콜린알포세레이트 성분의 연한 노란색 타원형 캡슐이라, 각인이 안 보이는 면이 찍히면 구분할 단서가 없다.



이 실험에서 처음으로 로컬 검증 점수와 Kaggle 점수가 반대로 움직였다. 검증 사진에서는 틀린 사례 두 건이 해결됐지만 Kaggle 점수는 0.007 떨어졌다. 이후 실험부터는 로컬 점수가 아닌 Kaggle 점수를 기준으로 판단했다.

6.5 cos_lr

가설 — 남은 문제는 위치가 아니라 확신이다. 한 알약에 두 답이 0.71과 0.70으로 비슷하게 남았으니, 학습 막바지에 세밀하게 다듬는 과정이 부족했다고 봤다. 후반 학습률을 코사인 모양으로 줄이면 마무리가 정밀해질 것으로 예상했다(3.4절).

조치 — cos_lr을 False에서 True로. 나머지는 그대로 뒀다.

결과 — 검증 점수 0.984 → **0.9784** / 제출 점수 0.40805 → **0.41550**

분석 — 결과는 가설과 달랐다. 검증 점수가 올라야 했는데 오히려 내려갔다.

원인은 검증 분류 손실에서 보인다. 두 방식 모두 91회째에 손실이 갑자기 올랐다. 이때 close_mosaic가 작동한다. 모자이크 증강은 사진 여러 장을 이어 붙여 학습시키는 방식인데, 마지막 10회에는 이를 끄고 평범한 사진 한 장씩을 보여준다. 입력이 갑자기 달라지니 모델이 다시 적응해야 한다.

방식	90회 (끄기 직전)	91회 (끈 직후)	100회 (최종)
선형 (기본)	0.375	0.390	0.379
cos_lr	0.382	0.431	0.445

차이는 회복 여부에서 갈렸다. 선형은 94회째 0.459까지 올랐다가 99회째 0.370으로 돌아왔지만, 코사인은 91회 이후 0.43~0.445에 머물렀다. 이 시점의 학습률은 선형이 0.000109, 코사인이 0.000034로 약 3배 차이였다. 코사인은 가중치를 고칠 수 있는 폭이 그만큼 작아서, 달라진 입력에 다시 맞추지 못한 것으로 보인다.

로컬 검증 점수는 모두 나빠졌지만 Kaggle 점수는 0.0075 올랐다. 6.4절과 정반대 결과로, 두 실험을 함께 보면 로컬 점수로 Kaggle 점수를 대신할 수 없다는 것이 분명해진다.

6.6 하이퍼파라미터의 한계

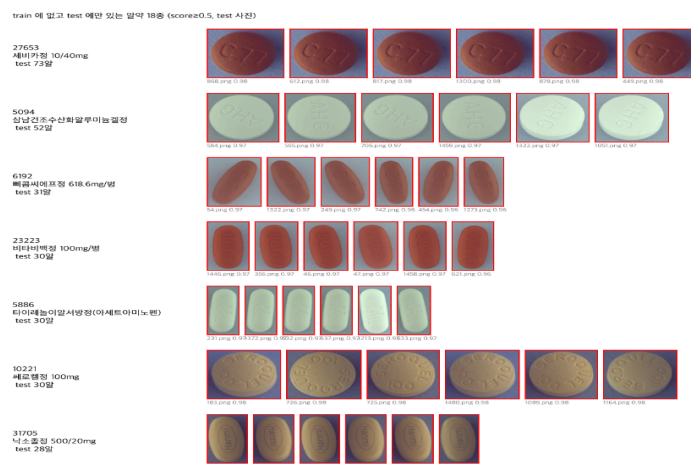
아래 표는 이렇게 한 가지씩만 따로 바꿔 측정한 결과다.

제출 점수	실험	바꾼 것	검증 점수
0.41738	해상도 1280	해상도	0.995
0.41550	cos_lr	학습률 스케줄	0.9784
0.41504	val 재분할	데이터 나누는 방식	0.984
0.41467	yolo26s	모델 크기	0.981
0.40516	freeze 10	전이학습	0.9738
0.40316	cls_pw 0.5	손실 가중치	0.9776
0.39132	mosaic 0.5	증강	0.976

일곱 가지를 모두 바꿔봤지만 제출 점수는 0.39132에서 0.41738 사이, 폭으로는 0.026 안에서만 움직였다. 순위만 바뀔 뿐 한계가 있었다. 같은 기간 검증 점수는 전부 0.97 이상이였다.

원인 분석 — 로컬에서는 거의 만점인데 제출은 0.4대라면, 정답을 모르는 무언가가 **test**에 있다는 뜻이다. **test** 사진을 직접 열어보니 **학습 데이터 56종에 없는 알약이 섞여 있었다**. 이후 AI Hub 데이터로 학습한 모델로 **test** 전체를 예측해 확인한 결과, 이런 알약이 18종이었다(아래 그림). 배운 적 없는 알약은 어떤 설정으로도 맞힐 수 없다. 6.3절의 33009와 비슷한 상황이지만, 그때는 데이터를 다시 나눠 해결할 수 있었고 이번에는 가진 데이터 안에 그 알약이 아예 없었다.

문제는 학습이 덜 된 것이 아니라 **알아야 할 정보가 없는 것이었다**. 손댈 곳은 학습 설정이 아니라 데이터였다.



(test에만 등장하는 알약 18종 중 7종)

6.7 AI Hub 데이터 추가

가설 — **test**에 모델이 학습하지 못하는 알약이 있다면, 그 알약들을 추가로 학습시키면 된다. 설정을 바꿔서는 만들 수 없는 정보를 데이터로 직접 넣는 것이다.

조치 — AI Hub 「경구약제 이미지 데이터」(datasetkey 576)의 조합 데이터를 학습 데이터에만 더했다(4.6절).

항목	이전	이후
알약 종류	56종	118종
학습 이미지	183장	10,394장
검증 이미지	37장	37장 (그대로)
학습 횟수	100회	30회

프로젝트 데이터의 원본인 TL_2_조합과 TS_2_조합은 제외했다. 데이터가 약 56배로 늘었으니 학습 횟수를 100회에서 30회로 줄이고 warmup_epochs와 close_mosaic도 같은 비율로 맞췄다.

결과 — 검증 점수 0.9784 → **0.9929** / 제출 점수 0.41550 → **0.61074**

0.195가 올랐다. 설정 일곱 가지를 전부 바꿔서 얻은 폭이 0.026이었으나 7.5배다. 6.6절의 진단이 맞았음을 확인했다.

관찰 — 알약별 점수를 보니 118종 대부분이 0.99 이상으로 안정적이었는데, 3832 하나만 0.932로 뒤처졌다.

6.8 오버샘플링 + WBF 앙상블

AI Hub를 도입한 뒤로는 실험이 두 갈래로 나뉘어 동시에 진행됐다. 이 절은 그중 최고 점수로 이어진 쪽이고, 출발점은 같은 데이터로 학습한 0.61084 구성이다.

오버샘플링

관찰 — 118종 중 3832 하나만 점수 0.932로 뒤처졌다. 나머지는 0.99 이상이었다.

가설 — 그 알약을 학습 중에 충분히 보지 못했다. 데이터를 더 구할 곳이 없으니 있는 사진을 더 자주 보여준다.

조치 — 데이터가 적은 알약(35206, 3832)의 사진을 하드링크로 2배 복제했다. 새 사진을 만든 게 아니라 같은 사진을 더 자주 보여주는 방식이다(4.7절).

결과 — 제출 점수 0.61084 → **0.61802**. 3832의 점수는 0.932에서 **0.995**로 회복했다.

다만 복제 배율을 3배로 더 올리고 대상 알약을 바꾼 실험에서는 오히려 학습이 제대로 되지 않았다. 더 자주 보여주는 것과 그 사진에만 익숙해지는 것 사이에 적정선이 있다는 걸 확인했다.

WBF 앙상블

가설 — 학습으로 더 얻을 게 없다면, 이미 가진 답을 더 잘 합치는 방법이 남아 있다. 서로 다른 조건으로 학습한 모델은 서로 다른 실수를 하니, 합치면 서로의 약점을 메워줄 것으로 봤다(3.3절).

조치 — 따로 학습한 두 모델의 예측을 WBF로 합쳤다. 얼마나 겹쳐야 같은 알약으로 볼지는 0.55, 합친 뒤 버릴 기준은 0.001로 뒀다.

항목	모델 A	모델 B
해상도	1280	960
배치	4	8
학습 횟수	50회	30회
오버샘플링	1배	2배 (35206·3832)
단독 제출 점수	0.61802	0.62041

결과

구성	제출 점수	단독 최고 대비
모델 B 단독	0.62041	—
앙상블, 비중 1 : 1	0.62054	+0.00013
앙상블, 비중 1 : 1.3	0.62550	+0.00509

분석 — 두 모델을 반반으로 합쳤을 때 얻은 건 +0.00013뿐이었다. 상대적으로 약한 모델 A의 오차가 그대로 섞여 들어 갔기 때문이다. 더 잘하는 모델 B 쪽에 비중을 실어 A를 보조로 쓰자 +0.00496이 더 붙었다. 합치는 것 자체보다 어떤 비율로 섞느냐가 성능을 갈랐다.

6.9 실패한 실험

점수를 올리지 못한 실험도 기록한다. 무엇이 원인이 아닌지를 좁혀줬기 때문이다.

실험	가설	결과
CLAHE 전처리	밝기 대비를 높이면 각인이 선명해진다	0.61084 → 0.6072 (되돌림)
멀티스케일 TTA + WBF	같은 모델로 해상도 크기만 바꿔 여러 번 예측해 합치면 좋아진다	0.61084 → 0.61060 (변화 없음)

실험	가설	결과
오버샘플링 3배	더 많이 복제하면 더 좋아진다	0.61084 → 0.60591 (하락)

CLAHE — 점수가 왜 떨어지는지 알 수 없어서 사진을 확대해 봤다. 알약 사진의 배경은 원래 매끈해서 밝기 차이가 거의 없는데, **CLAHE**가 타일(8×8)마다 대비를 높이면서 원래 없던 얼룩이 생겼다. 그 배경 부분의 픽셀 표준편차가 3.3에서 6.7로 두 배 이상 거칠어졌고, 모델이 이 얼룩을 알약 경계로 잘못 인식했다.

멀티스케일 TTA — 같은 모델에 입력 크기(960·1280)와 좌우반전만 바꿔 세 번 예측하고 **WBF**로 합치는 방식이다. 하지만 모델은 학습 중에 이미 크기 변형(±50%)과 좌우반전 증강을 거쳤기 때문에, 세 예측은 거의 같게 나온다. 같은 곳에서 틀리는 예측끼리는 합쳐도 새로운 정보가 없다.

이는 해상도, 배치, 학습 횟수를 다르게 해 따로 학습한 두 모델을 합친 6.8절의 결과(0.62550)와 대비된다. 앙상블의 효과는 합치는 행위 자체가 아니라, 모델들이 서로 다른 곳에서 틀린다는 데서 나온다.

오버샘플링으로 3832은 개선됐지만, 대신 18147의 점수가 0.931로 떨어졌다. 특정 알약을 더 자주 보여주면 그만큼 다른 알약의 학습 비중이 줄어들기 때문이다.

7. 논의

7.1 튜닝의 문제가 아니라 데이터의 한계였다

6.6절과 6.7절만 보면 "데이터가 설정보다 중요하다"는 결론으로 읽히기 쉽다. 하지만 데이터를 늘린 뒤에도 설정 조정은 계속 점수를 올렸고, 오히려 더 크게 올렸다.

구간	시작	끝	얻은 폭
AI Hub 이전 설정 조정	0.39132	0.41738	+0.026
AI Hub 이후 설정 조정·구성	0.59	0.62550	+0.036

정확히 말하면 이렇다. 데이터가 부족하면 설정을 아무리 손봐도 올라갈 수 있는 한계가 낮다. 같은 노력을 들여도 얻는 폭이 데이터 양에 묶인다. 그러니 문제는 "설정이나 데이터냐"의 선택이 아니라 순서였다. 데이터를 먼저 확보하고 설정을 조정했다면 같은 시간으로 더 많이 얻었을 것이다.

7.2 로컬 검증 점수는 판단 기준이 되지 못했다

두 가지 측면이 있다.

차이가 크다 — 로컬 검증 점수는 줄곧 0.97~0.99였는데 제출 점수는 0.4대였다. 이유는 검증 데이터의 구성에 있다. 검증은 37장이고, 4.5절에서 모든 알약이 학습에 들어가도록 보정했기 때문에 검증에는 모르는 알약이 아예 없다.

방향까지 반대다 — 두 실험이 정반대로 움직였다.

실험	로컬 점수	제출 점수
해상도 1280 (6.4)	0.984 → 0.984	0.41503 → 0.40805 (하락)
cos_lr (6.5)	0.984 → 0.9784 (하락)	0.40805 → 0.41550 (상승)

다만 둘이 아무 상관없다고까지는 말할 수 없다. 6.6절의 일곱 실험을 비교해 보면, 로컬 검증에서 더 나은 실험이 Kaggle에서도 대체로 더 나왔다. 하지만 점수 자체는 크게 달랐고, 그 차이를 만든 것은 test에만 있는 알약이었다. 이 사실을 확인한 뒤로는 모든 판단을 Kaggle 점수로 내렸다.

7.3 데이터 수준의 미세조정에는 한계가 있다

오버샘플링으로 3832의 점수를 0.932에서 0.995로 끌어올렸지만, 그 대신 18147이 0.931로 떨어졌다. 한 알약을 더 자주 보여주면 다른 알약의 학습 비중이 줄어들기 때문이다. 이처럼 점수가 낮은 알약을 하나씩 골라 보완하는 방식은 한쪽을 올리면 다른 쪽이 내려가 한계가 있다. 근본적인 해결책은 특정 알약만 강화하는 것이 아니라 전체 데이터를 늘리는 것이다.

7.4 실패한 실험의 역할

AI Hub 이전의 일곱 실험(6.6절)은 어느 것도 0.42를 넘지 못했다. 하지만 해상도, 학습률, 손실, 증강, 모델 크기를 하나씩 지워간 덕분에 문제가 학습 방식에 있지 않다는 결론에 이를 수 있었다. 이 과정이 없었다면 데이터로 방향을 틀 근거도, 0.195의 개선도 없었을 것이다.

특히 멀티스케일 TTA의 실패는 6.8절 앙상블의 성공과 짝을 이룬다. 같은 모델을 여러 크기로 돌려 합친 결과는 변화가 없었고(0.61084 → 0.61060), 서로 다른 조건으로 학습한 두 모델을 합친 결과는 0.62550이었다. 앙상블이 왜 효과를 내는지를 두 실험을 비교하면 드러난다.

7.5 남은 실패 유형

최종 모델이 검증에서 틀리는 사례를 보면 알약 이름을 헛갈리는 경우는 거의 없다. 남은 건 두 가지다.

1. **중복 예측** — 한 알약에 박스가 둘. WBF가 하나로 합치지 못한 경우
2. **배경 오염** — 사진 가장자리 빈 곳에 알약이 없는데 그려진 박스

두 사례 모두 알약 이름은 맞혔고, 그 옆에 신뢰도가 매우 낮은(0.00~0.01) 박스가 하나씩 더 남아 있을 뿐이다. mAP는 신뢰도가 높은 박스부터 차례로 채점하기 때문에, 이렇게 신뢰도가 낮은 박스는 점수에 거의 영향을 주지 않는다.

2.4절에서 설명했듯이 제출할 때는 신뢰도 0.001 이상인 박스를 모두 포함했다. 그래서 결과 그림은 지저분해 보이지만, 낮은 박스를 버리면 혹시 맞았을 박스까지 잃게 되므로 점수 면에서는 남겨 두는 편이 유리하다고 판단했다.

결국 이 과제에서 마지막까지 남은 어려움은 알약 이름을 맞추는 것이 아니라, 알약 위치를 얼마나 정확하게 맞히느냐였다.

7.6 비즈니스 시사점

지금 성능이 어느 정도인가 — 최종 점수 0.62550은 박스를 정확히 그려야만 인정하는 기준에서 나온 값이다. 복약 확인처럼 "무슨 약인지"가 중요한 용도에서는 박스의 정확도 기준이 이보다 느슨하므로 실제 체감 정확도는 더 높다. 실제로 최종 모델이 알약 이름을 헛갈리는 경우는 118종 대부분에서 거의 없었다.

서비스로 만들려면 무엇이 더 필요한가 — 크게 두 가지를 보완해야 한다.

첫째, 신뢰도 기준을 서비스에 맞게 0.25 정도로 높여 오탐을 줄여야 한다. 지금은 점수를 위해 기준을 0.001로 낮춰 두었기 때문에, 이대로 쓰면 배경이나 알약이 없는 자리에도 박스가 표시된다.

둘째, 배우지 않은 알약에 대해 "미등록 알약"이라고 답하는 기능이 필요하다. 지금 모델은 118종 중 하나를 반드시 고르기 때문에, 처음 보는 알약에도 틀린 이름을 그대로 내놓는다. 알약을 다루는 서비스에서 이는 단순한 오답보다 문제가 된다.

데이터를 얼마나 모으느냐가 곧 서비스 범위다 — AI Hub 데이터를 추가해 알약을 56종에서 118종으로, 학습 사진을 183장에서 10,394장으로 늘리자 점수가 0.195 올랐다. 특히 test에만 있던 알약을 학습하게 된 것이 컸다. 즉, 서비스가 알아볼 수 있는 알약의 범위는 확보한 데이터의 범위를 넘을 수 없다. 국내에 유통되는 경구용 알약이 매우 많다는 점을 생각하면, 모델을 더 다듬는 것보다 데이터를 꾸준히 모으는 체계를 만드는 것이 더 실질적인 과제다.

8. 결론 및 향후 과제

8.1 결론

최종 구성은 AI Hub 데이터를 합친 118종·10,394장으로 학습한 YOLO26n 두 개를 WBF로 합친 것이고, 제출 점수 **0.62550**을 기록했다. 처음 0.235552에서 2.65배가 됐다.

최종 학습 설정은 다음과 같다. 각 값을 왜 그렇게 정했는지는 6장에 있다.

```
{
  "optimizer": "AdamW",
  "lr0": 0.001,
  "cos_lr": True,
  "warmup_epochs": 1.5,
  "close_mosaic": 3,
  "box": 12.0,
  "cls": 1.0,
}
```

box와 cls는 박스 위치 오차와 이름 오답에 주는 벌점의 크기다. 이 프로젝트는 IoU 0.75 이상에서만 점수를 주고 이름이 틀려도 점수를 잃으므로, 두 값을 모두 기본값(box 7.5, cls 0.5)보다 키웠다(box 12.0, cls 1.0). close_mosaic은 학습 마지막 3회 동안 모자이크 증강을 끄는 설정인데, 증강이 없는 실제 상황에 맞춰 마무리하려는 것이다.

구간별로 얼마나 올랐는지는 다음과 같다.

구간	점수 변화	얻은 폭
기본 설정 → 설정 조정	0.235552 → 0.41550	+0.180
AI Hub 데이터 통합	0.41550 → 0.61074	+0.195
오버샘플링 + WBF 앙상블	0.61074 → 0.62550	+0.015

앞서 던진 질문에 대한 답은 이렇다. 성능을 가로막은 것은 학습 설정이 아니라 데이터였다. 다만 7.1절에서 정리했듯 설정 조정이 무의미하지 않았다. 데이터가 부족한 동안에는 도달할 수 있는 영역이 한계가 있었다.

8.2 배운 것

1. **데이터는 직접 눈으로 봐야 한다.** 사진을 하나씩 열어보고서야 **test**에만 있는 알약, 생김새가 같고 각인만 다른 알약, 앞뒷면으로 찍혀 각인이 안 보이는 사진이 있다는 걸 알았다. 표와 숫자만 봤다면 끝까지 몰랐을 것들이다.
2. **모델도 여러 버전을 써봤어야 했다.** 세 모델을 비교해 YOLO26n을 고른 뒤로는 대부분의 실험을 이 하나로 돌렸다. 정작 마지막에 점수를 올린 건 다르게 학습한 모델 둘을 합친 앙상블이었다. 모델 자체를 더 다양하게 가져갔다면 폭이 더 컸을 것이다.
3. **로컬 점수를 믿으면 안 된다.** 제출 점수와 절대 수준이 다르고, 방향까지 뒤집힌 사례도 있었다. 검증 데이터가 실제 시험의 어려움을 담고 있지 않으면 그 점수는 참고용에 그친다.
4. **기록을 남기는 것이 곧 분석이다.** 모든 실험이 자동으로 기록됐기에 일곱 가지 결과를 한 표에 놓고 비교할 수 있었고, 그 표를 보고서야 **0.41**의 벽을 발견했다. 기록이 흩어져 있었다면 같은 실험을 반복하면서도 벽의 존재를 몰랐을 것이다.

8.3 향후 과제

과제	내용	우선순위
데이터 추가 확장	AI Hub에 아직 쓰지 않은 단독·조합 데이터가 남아 있음	높음
3개 모델 앙상블	모델을 하나 더 추가하고 WBF 설정을 다듬기.	중간
모르는 알약 거부	배우지 않은 알약에 "미등록 알약"이라고 답하는 기능. 프로젝트 점수에는 무의미하나 서비스에는 필수	낮음
노이즈 걸러내기	가장자리 배경 박스 제거. 점수에는 영향 없고 사용성에만 관련	낮음

검증 데이터에서 대부분의 알약 점수가 이미 **0.99**에 도달해 있어, 하이퍼파라미터를 다시 조정해도 얻을 수 있는 이득은 크지 않다. 남은 어려움은 이름을 맞추는 것이 아니라 박스를 정확히 그리는 일이다. 이 부분은 데이터를 늘리고 앙상블 구성을 다듬는 방향으로 접근하는 편이 현실적이다.

9. 참고문헌

- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. ICCV.
- Solovyev, R., Wang, W., & Gabruseva, T. (2021). Weighted boxes fusion: Ensembling boxes from different object detection models. Image and Vision Computing, 107, 104117.
- Ultralytics. YOLO Documentation. <https://docs.ultralytics.com>
- AI Hub. 경구약제 이미지 데이터 (datasetkey 576). <https://aihub.or.kr>

10. 소감

엄재웅 (팀장)

이번 프로젝트는 **VS Code**로 진행한 첫 프로젝트였습니다. 가상환경을 맞추고, 브랜치를 나누고, 서로의 코드를 합치는 과정이 모두 낯설어서 초반에는 모델보다 팀원들과 호흡을 맞추는 데 더 많은 시간을 썼습니다. 하지만 그때 협업 방식을 함께 정리해 둔 덕분에 후반으로 갈수록 작업 속도가 빨라졌고, 각자 맡은 부분을 병렬로 진행할 수 있었습니다. **GitHub** 협업 익히기, **VS Code**에 익숙해지기, 끝까지 마무리하기라는 처음 목표를 모두 이뤄 만족합니다.

가장 기억에 남는 것은 팀원들의 태도였습니다. 기간 내내 한 명도 결석하지 않고 매일 모여 진행 상황을 공유했고, 누군가 막히면 다른 팀원이 자연스럽게 도왔습니다. 혼자였다면 방향을 잃었을 순간도 팀이 함께였기에 넘길 수 있었습니다.

실험 과정도 의미 있었습니다. 설정을 무작위로 바꿔 보는 대신, 틀린 사례를 보고 왜 틀렸는지 가설을 세운 뒤 한 가지만 바꿔 확인하는 방식으로 진행했습니다. 그렇게 하나씩 원인을 지워 간 끝에 문제가 학습 설정이 아니라 데이터에 있다는 결론에 스스로 도달했고, 점수가 크게 오른 것보다 그 과정을 직접 봐야겠다는 점이 더 값졌습니다.

아쉬운 점은 방향을 정하는 단계에서 제 의견이 크게 반영되다 보니, 팀원들에게 어느 정도 강제성을 준 것은 아니었나 하는 점입니다. 다음 프로젝트에서는 큰 그림을 팀원들과 함께 그리고, 각자가 제안한 방향도 충분히 시도해 볼 수 있는 분위기를 만들고 싶습니다. 끝까지 함께해 준 팀원들에게 진심으로 감사드립니다.

신재민

이번 프로젝트를 통해 모델의 크기를 키우거나 학습 횟수를 늘리는 것만으로는 항상 성능이 향상되는 것은 아니라는 점을 배웠습니다. 또한 데이터의 양과 품질이 모델 성능에 큰 영향을 준다는 것을 느꼈으며, 부족한 데이터를 추가하고 데이터 구성을 개선하는 것도 중요한 방법이라는 것을 알게 되었습니다. 실험 과정에서는 데이터 분할과 학습 조건을 동일하게 유지해야 정확한 성능 비교가 가능하다는 점을 깨달았고, 특히 모델이 자주 틀리는 부분을 분석하고 그 원인을 찾아 개선하는 과정이 중요하다는 것을 경험했습니다. 이번 프로젝트를 통해 단순히 높은 점수를 얻는 것보다 실험 결과를 분석하고 성능이 달라진 이유를 이해하는 것이 더 중요하다는 점을 배웠습니다.

김라희

이번 프로젝트를 통해 성능 향상에 있어 체계적인 실험기록과 이력관리의 중요성을 깊이 체감했습니다. 모델의 규모를 키우거나 파라미터에 쏟는 시간 대비 양질의 데이터셋을 체계적으로 확충하는 작업이 실제 성능개선에 훨씬 더 결정적인 영향을 미친다는 점을 명확히 실감했습니다. 시행 착오를 거치며 얻은 이러한 인사이트는 향후 실제 도메인에서 데이터 기반의 문제 해결 역량을 다지는데 소중한 자산이 될거라 기대하고있습니다. 아울러 훌륭한 팀원들과 상호 보완하며 소통하는 과정속에서 각자의 장점으로 한계를 극복하고 시너지를 창출하는 협업의 가치를 깊이 깨닫는 뜻깊은 시간이었습니다.

윤성원

이번 프로젝트를 진행하면서 **ai** 모델을 만드는 과정은 단순히 좋은 모델을 선택하고 학습시키는 것만으로 끝나는 것이 아니라는 것을 느꼈습니다. 처음에는 모델의 성능을 높이기 위해 하이퍼파라미터를 조정하고 학습 조건을 바꾸는 것이 가장 중요하다고 생각했습니다. 하지만 수차례 실험을 진행하면서 모델의 설정을 바꾸는 것만으로는 성능 향상에 한계가 있다는 것을 직접 경험했습니다.

특히 데이터를 직접 확인하면서 학습 데이터에 없는 알약이 테스트 데이터에 포함되어 있다는 것을 발견했고, 색과 모양은 비슷하지만 각인으로 구분해야 하는 알약도 있다는 것을 알게 되었습니다. 이를 통해 모델이 아무리 좋은 설정으로 학습하더라도 학습할 정보 자체가 부족하면 해결할 수 없는 문제가 있다는 것을 이해하게 되었습니다. 이후 **ai Hub** 데이터를 추가하면서 성능이 크게 향상되는 것을 보며, 모델을 어떻게 학습시킬 것인가만큼 어떤 데이터를 가지고 학습 할 것인가도 중요하다는 것을 배웠습니다.

여러 실험의 결과를 기록하고 비교하는 과정도 인상 깊었습니다. 처음에는 각각의 실험이 단순히 점수를 높이기 위한 과정이었지만, 결과를 모아 비교해보니 어떤 방법이 효과가 있었고 어떤 방법은 큰 차이가 없었는지 보였습니다. 결국 실험 기록 자체가 다음 방향을 결정하기 위한 중요한 자료가 된다는 것도 이번 프로젝트를 통해 배웠습니다.

개인적으로는 **Part 1**과 **Part 2**에서 배웠던 **Python**과 머신러닝, 딥러닝 관련 내용들이 실제 프로젝트에서 어떻게 사용되는지 연결해서 볼 수 있었던 점이 가장 의미 있었습니다. 수업에서 배울 때는 각각의 개념이

따로 느껴졌지만, 실제로 데이터를 정리하고 모델을 학습하고 결과를 확인하는 과정을 거치면서 서로 연결되어 있다는 것을 조금씩 이해할 수 있었습니다.

무엇보다 이번 프로젝트는 혼자 공부할 때보다 팀으로 문제를 해결하는 과정에서 더 많은 것을 보고 배울 수 있다는 것을 느꼈습니다. 부족한 부분이 많았지만 팀원들과 함께 질문하고 의견을 나누면서 하나의 결과물을 완성할 수 있었습니다. 아직도 여전히 모르는 것이 많지만, 이번 경험을 통해 직접 데이터를 확인하고, 실험하고, 결과를 비교하면서 하나씩 해결해 나가는 것이 중요하다는 것을 재차 배웠습니다.

이번 프로젝트는 단순히 YOLO 모델을 학습시키는 경험을 넘어, ai 프로젝트에서 문제를 발견하고 데이터를 확인하고 실험을 통해 해결 방법을 찾아가는 과정 자체를 경험했다는 점에서 의미가 있었습니다.

마지막으로, 파이리 팀원들과 함께 첫 프로젝트를 진행할 수 있었던 것에 감사하며, 좋은 팀원들과 함께 프로젝트를 완성할 수 있어 더욱 뜻깊은 시간이었습니다.